

Installer Jupyter

Guide pas à pas — macOS, Linux, Windows
sans rien installer dans le Python de votre système

Joseph Azar — Maître de Conférences
UMLP — Université Marie et Louis Pasteur
Chercheur FEMTO-ST
joseph.azar@umlp.fr

ELC2 — Introduction au machine learning — UFR STGI
Version d'octobre 2026

À la fin de ce guide, vous aurez sur votre ordinateur un dossier `ia-elc2` qui contient son propre Python et les bibliothèques du cours, et Jupyter ouvert dans votre navigateur sur votre premier notebook.

Aucun compte à créer, aucun mot de passe, aucun droit administrateur. Vous n'avez pas besoin d'avoir déjà Python.

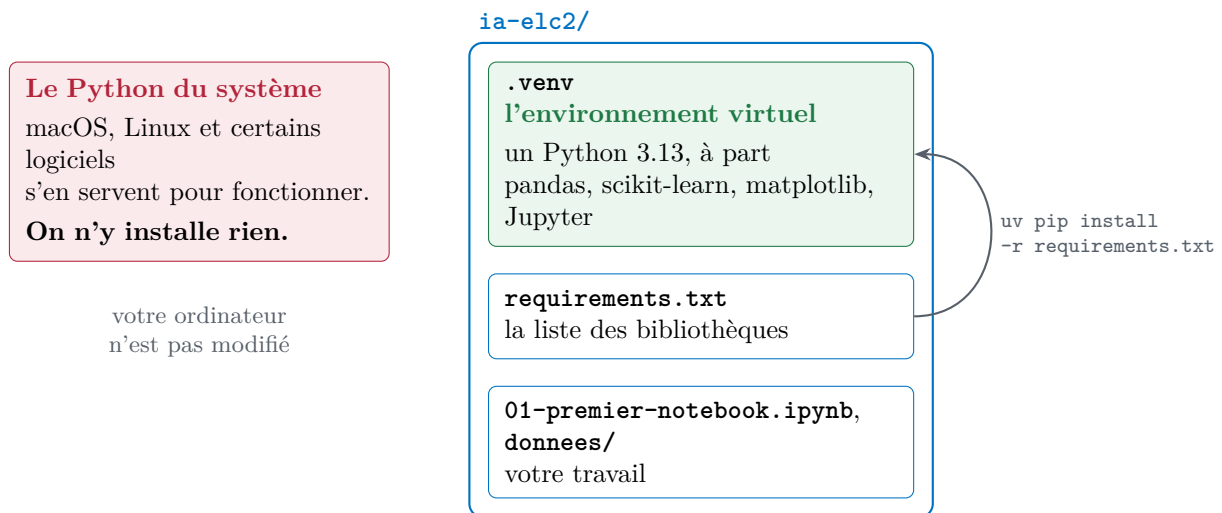
Lisez les deux premières pages avant de taper quoi que ce soit. Elles expliquent *pourquoi* on installe de cette façon. Ensuite, suivez les étapes dans l'ordre, sans en sauter. Si une étape ne donne pas le résultat annoncé, arrêtez-vous et regardez le tableau « Problèmes fréquents » à la fin.

Les mots à connaître

Mot	Ce que c'est
Terminal	Une fenêtre où l'on tape des commandes au lieu de cliquer. Vous tapez une ligne, vous appuyez sur Entrée , l'ordinateur l'exécute.
Python	Le langage de programmation le plus utilisé pour l'IA et l'analyse de données.
Bibliothèque	Une boîte à outils écrite par d'autres, que l'on ajoute à Python : pandas (des tableaux, comme Excel), scikit-learn (le machine learning), matplotlib (les graphiques).
Python du système	Le Python déjà présent sur votre ordinateur, ou installé un jour par un autre logiciel. On n'y installe rien.
Environnement virtuel	Un Python à part, rangé dans le dossier du cours, avec ses propres bibliothèques. Son dossier s'appelle <code>.venv</code> .
requirements.txt	Un fichier texte : la liste des bibliothèques du cours et de leurs versions.
Jupyter, notebook	Un cahier qui s'ouvre dans le navigateur, où l'on mélange texte, code et résultats. Un notebook est un fichier <code>.ipynb</code> .
uv	Le seul programme que vous installez vous-même. C'est lui qui crée l'environnement virtuel et y installe tout le reste.

Le principe : une trousse par cours

Imaginez que chaque cours ait sa trousse. Dans la trousse du cours d'IA, il y a les outils de ce cours, dans les bonnes versions. Vous ne mélangez pas ces outils avec ceux de votre ordinateur, et vous ne mélangez pas les trousse entre elles.



Pourquoi ne pas installer directement dans le Python du système ?

- **Il ne vous appartient pas vraiment.** Sur macOS et Linux, le système et d'autres logiciels s'en servent. Y ajouter ou y changer une bibliothèque peut les casser. C'est pourquoi les versions récentes de Linux et de macOS refusent souvent, avec une erreur `externally-managed-environment`.
- **Les versions se contredisent.** Ce cours veut pandas 3.0 ; un autre cours ou un stage voudra peut-être pandas 2. Un environnement par dossier, et chacun a la sienne.
- **On peut tout effacer proprement.** Supprimez le dossier `ia-elc2` : les bibliothèques du cours partent avec lui. Rien ne traîne ailleurs.

Le fichier `requirements.txt` : la liste des courses

Il est dans le dossier du cours. Ouvrez-le avec n'importe quel éditeur de texte, il ressemble à ceci :

```
requirements.txt

# Bibliothèques du cours ELC2 - Introduction au machine learning.
# A installer DANS l'environnement virtuel du dossier ia-elc2 :
#     uv pip install -r requirements.txt
notebook==7.6.3
pandas==3.0.6
scikit-learn==1.9.1
matplotlib==3.11.2
openpyxl==3.1.5
```

- Une ligne par bibliothèque. `==` veut dire « exactement cette version ».
- Les lignes qui commencent par `#` sont des commentaires : elles ne sont pas lues.
- `notebook`, c'est Jupyter. `openpyxl` sert à lire les fichiers Excel.

Tout le monde installe la même liste, donc tout le monde a les mêmes versions. Un code qui marche chez votre voisin marchera chez vous. Le fameux « ça marche chez moi » disparaît.

Ce qu'il ne faut jamais faire

Vous trouverez sur Internet des tutoriels qui disent de taper ces commandes. **Pas pour ce cours.**

À ne pas taper	Pourquoi
<code>sudo pip install ...</code>	installe dans le Python du système, avec les droits administrateur
<code>pip install ...</code>	dans un terminal quelconque, on ne sait pas dans quel Python ça va
<code>pip install --break-system-packages</code>	le nom dit tout : on force le système à accepter

Dans le dossier du cours, la bonne commande est `uv pip install`. Et `uv` vous protège : s'il ne trouve pas d'environnement virtuel, il **refuse** d'installer, plutôt que de toucher au système :

```
error: No virtual environment found; run `uv venv` to create an environment, or
pass `--system` to install into a non-virtual environment
```

Si vous voyez ce message, `uv` vous a évité une erreur. Vérifiez que vous êtes dans `ia-elc2` et que l'étape 4 a été faite.

Étape 1 — Ouvrir un terminal

Système	Comment faire
macOS	Cmd + Espace, tapez Terminal , puis Entrée.
Windows	Touche Windows , tapez PowerShell , puis Entrée. Sous Windows 11, l'application Terminal convient aussi.
Linux	Ctrl + Alt + T (Ubuntu), ou cherchez « Terminal » dans les applications.

Une fenêtre s'ouvre avec une ligne qui se termine par %, \$ ou > : c'est l'**invite**, elle attend votre commande.

Copiez-collez les commandes de ce guide une ligne à la fois, puis appuyez sur Entrée. Attendez que l'invite revienne avant de coller la ligne suivante. Pour coller : Cmd + V sur macOS, Ctrl + V sous Windows, Ctrl + Maj + V dans un terminal Linux.

Étape 2 — Installer uv

Sur macOS et sur Linux

```
curl -LsSf https://astral.sh/uv/install.sh | sh
```

Sous Linux, si le terminal répond `curl: command not found`, utilisez `wget` :

```
wget -qO- https://astral.sh/uv/install.sh | sh
```

Sur Windows, dans PowerShell

```
powershell -ExecutionPolicy ByPass -c "irm https://astral.sh/uv/install.ps1 | iex"
```

Copiez **la ligne entière**, guillemets compris. `-ExecutionPolicy ByPass` autorise ce script précis à s'exécuter, une seule fois, sans changer les réglages de Windows.

uv s'installe dans votre dossier personnel, pas dans le système. C'est pour cela qu'il n'a besoin d'aucun mot de passe administrateur.

Recharger le terminal

Fermez complètement la fenêtre du terminal, puis ouvrez-en une nouvelle. C'est l'étape que tout le monde oublie : tant que le terminal n'a pas été rouvert, il ne connaît pas la commande `uv`.

Vérifier

Dans le **nouveau** terminal :

```
uv --version
```

Vous devez voir un numéro de version, par exemple :

```
uv 0.12.21
```

Étape 3 — Placer le dossier du cours

Sur la page GitHub du cours, github.com/josephazar/ELC2-Introduction-IA-2026, ouvrez le dossier `seance-1`, cliquez sur `seance1-etudiants.zip`, puis sur le bouton de téléchargement (la flèche vers le bas, en haut à droite). Décompressez le fichier (double-clic sur macOS et Linux, clic droit → **Extraire tout** sous Windows). Vous obtenez un dossier `ia-elc2`. **Déplacez-le dans votre dossier personnel :**

Système	Votre dossier personnel
macOS	Finder → menu Aller → Départ (Cmd + Maj + H). C'est le dossier à votre nom.
Windows	C:\Utilisateurs\votre-nom. Dans l'Explorateur, tapez %USERPROFILE% dans la barre d'adresse, puis Entrée.
Linux	Fichiers → Dossier personnel .

Sous Windows, Extraire tout crée un dossier `seance1-etudiants` qui contient `ia-elc2`. C'est `ia-elc2` qu'il faut déplacer, pas son parent.

Vérifier, dans le terminal

```
cd ~/ia-elc2
ls
```

`cd` veut dire *change directory*, entrer dans un dossier. `ls` affiche son contenu. Vous devez voir ces trois noms (sous Windows, dans un tableau) :

```
01-premier-notebook.ipynb  donnees  requirements.txt
```

À partir d'ici, toutes les commandes se tapent dans le dossier `ia-elc2`. Si vous ouvrez un nouveau terminal, recommencez par `cd ~/ia-elc2`.

Étape 4 — Créer l'environnement virtuel

```
uv venv --python 3.13
```

`uv` crée le sous-dossier `.venv` : la trousse, vide pour l'instant, avec un Python 3.13. Si aucun Python 3.13 n'est sur votre machine, `uv` en télécharge un et le range à part, dans ses propres dossiers. Le terminal affiche :

```
Using CPython 3.13.13
Creating virtual environment at: .venv
Activate with: source .venv/bin/activate
```

La dernière ligne vous propose d'« activer » l'environnement. Inutile : ignorez-la. Nous utiliserons `uv run`, qui fait la même chose automatiquement (voir l'encadré de l'étape 6).

Le dossier `.venv` est caché : son nom commence par un point. C'est normal si vous ne le voyez pas dans le Finder ou l'Explorateur. N'y touchez pas.

Étape 5 — Installer les bibliothèques du cours

```
uv pip install -r requirements.txt
```

`-r` veut dire « lis la liste dans ce fichier ». `uv` télécharge chaque bibliothèque de la liste, et celles dont elles ont besoin, soit une centaine de mégaoctets, et les range **dans** `.venv`. Le terminal se termine par une ligne par paquet, précédée d'un `+` :

```
Resolved 109 packages in 2.35s
Installed 109 packages in 1.05s
+ matplotlib==3.11.2
+ notebook==7.6.3
+ pandas==3.0.6
+ ...
```

Ce qui compte : l'absence du mot **error**. Le nombre de paquets peut varier légèrement.

Étape 6 — Lancer Jupyter

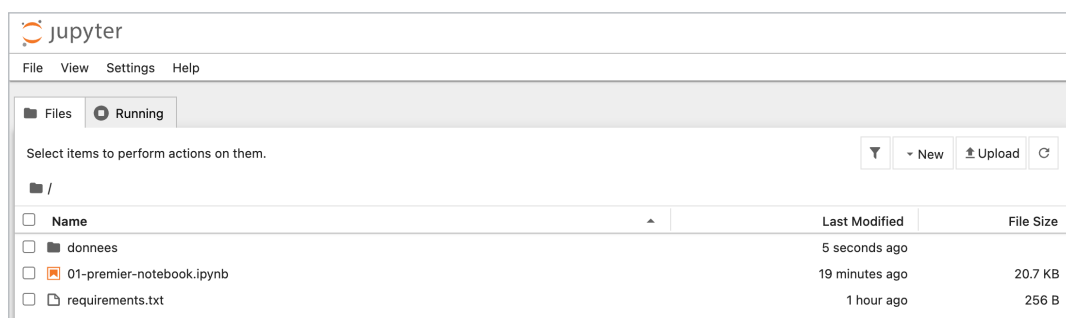
```
uv run jupyter notebook
```

`uv run` veut dire : **exécute cette commande avec le Python de `.venv`**. C'est ce qui garantit que Jupyter utilise les bibliothèques du cours, et pas autre chose.

Le terminal affiche plusieurs lignes, dont :

```
Jupyter Server 2.21.1 is running at:
http://localhost:8888/tree?token=3f9c1e...
Use Control-C to stop this server and shut down all kernels (twice to skip
confirmation).
```

Votre navigateur s'ouvre tout seul sur le contenu du dossier `ia-elc2` :

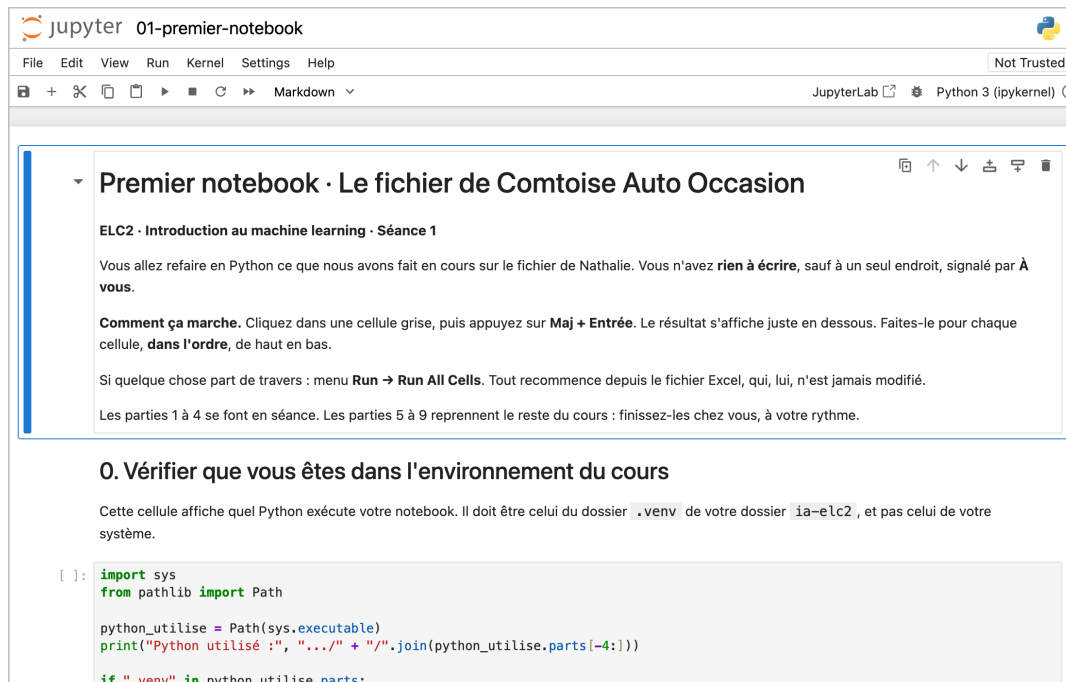


Ne fermez pas le terminal tant que vous travaillez. C'est lui qui fait tourner Jupyter. Réduisez-le, ne le fermez pas. `localhost` désigne votre propre ordinateur : vos fichiers ne partent pas sur Internet.

Et « activer l'environnement » ? Beaucoup de tutoriels commencent par `source .venv/bin/activate` (macOS, Linux) ou `.venv\Scripts\activate` (Windows). Cela fait utiliser `.venv` à tout le terminal, jusqu'à sa fermeture. `uv run` fait la même chose, pour une seule commande. Vous n'avez pas besoin des deux.

Étape 7 — Le premier notebook

Cliquez sur `01-premier-notebook.ipynb`. Il s'ouvre dans un nouvel onglet :



Cliquez dans la première cellule grise, celle de la partie « 0. Vérifier que vous êtes dans l'environnement du cours », puis appuyez sur **Maj + Entrée**. Vous devez obtenir :



Vous voyez « OK : ce notebook tourne dans l'environnement virtuel du cours » ? Votre installation est terminée, et elle est propre. Continuez le notebook, cellule après cellule, avec **Maj + Entrée**.

Vous voyez « ATTENTION : ce n'est pas le Python du cours » ? Jupyter a été lancé avec un autre Python, par exemple celui d'Anaconda. Fermez l'onglet, arrêtez Jupyter (**Ctrl + C** deux fois dans le terminal), vérifiez que les étapes 4 et 5 ont bien été faites dans `ia-elc2`, puis relancez `uv run jupyter notebook`.

Les gestes de base dans Jupyter

Geste	Effet
Maj + Entrée	Exécute la cellule et passe à la suivante. Le geste que vous ferez le plus souvent
[] : à gauche d'une cellule	Pas encore exécutée
[*] :	En cours d'exécution. Attendez
[3] :	Exécutée, en troisième position depuis l'ouverture du notebook
Menu Run → Run All Cells	Exécute tout le notebook, de haut en bas
Menu Kernel → Restart Kernel	Repart de zéro, quand plus rien ne répond
Cmd + S ou Ctrl + S	Enregistre. Jupyter enregistre aussi tout seul, régulièrement

L'ordre d'exécution compte, pas l'ordre à l'écran. Si vous exécutez directement la cinquième cellule, Python ne connaît pas encore ce que définissent les quatre premières : `NameError: name 'pd' is not defined`. Le remède : **Run** → **Run All Cells**.

Arrêter, puis revenir la semaine prochaine

Arrêter : enregistrez, revenez dans le terminal, appuyez **deux fois** sur **Ctrl + C** (sur macOS aussi : **Ctrl**, pas **Cmd**). Puis fermez le terminal.

Revenir : rien à réinstaller. Ouvrez un terminal et tapez deux lignes :

```
cd ~/ia-elc2
uv run jupyter notebook
```

Un nouveau codelab : déposez le notebook et ses données dans **ia-elc2**, ou utilisez le bouton **Upload** de la page d'accueil de Jupyter.

Une nouvelle liste : si une séance fournit un nouveau **requirements.txt**, remplacez l'ancien et relancez la même commande : `uv` n'installe que ce qui manque.

```
uv pip install -r requirements.txt
```

Problèmes fréquents

Ce que vous voyez	Cause	Solution
<code>command not found: uv</code> , ou sous Windows « <code>uv</code> n'est pas reconnu... »	Le terminal n'a pas été rouvert après l'installation	Fermez le terminal, ouvrez-en un nouveau
<code>cd : no such file or directory</code> , ou « Impossible de trouver le chemin »	Le dossier ia-elc2 n'est pas dans votre dossier personnel	Refaites l'étape 3 : il est peut-être resté dans Téléchargements
<code>error: File not found: 'requirements.txt'</code>	Vous n'êtes pas dans ia-elc2 , ou dans un dossier ia-elc2 en double	<code>cd ~/ia-elc2</code> , puis <code>ls</code> : vous devez voir requirements.txt
<code>error: No virtual environment found</code>	L'étape 4 n'a pas été faite, ou pas dans ce dossier	Dans ia-elc2 : <code>uv venv --python 3.13</code>

Ce que vous voyez	Cause	Solution
error: Failed to spawn: 'jupyter'	L'étape 5 n'a pas été faite, ou pas dans ce dossier	Dans ia-elc2 : <code>uv pip install -r requirements.txt</code>
La cellule 0 affiche ATTENTION	Jupyter a été lancé avec un autre Python (Anaconda, par exemple), souvent parce que l'étape 5 manque	Arrêtez Jupyter, faites l'étape 5 dans ia-elc2, relancez avec <code>uv run</code>
FileNotFoundError: ...ventes-3-agences.xlsx	Jupyter a été lancé depuis un autre dossier	Arrêtez-le, cd ~/ia-elc2, relancez
Le navigateur ne s'ouvre pas	Rien de grave	Dans le terminal, copiez l'adresse qui commence par <code>http://localhost:8888</code> et collez-la dans le navigateur
« Connexion impossible » sur localhost	Le terminal a été fermé	Relancez Jupyter (deux lignes, ci-dessus)
NameError: name '...' is not defined	Une cellule du haut n'a pas été exécutée	Run → Run All Cells
Une cellule reste sur [*]	Le calcul est bloqué	Kernel → Restart Kernel , puis Run All Cells
No module named pip après %pip install	Normal : on n'installe pas depuis le notebook	Dans le terminal, dans ia-elc2 : <code>uv pip install</code> suivi du nom, et ajoutez-le à <code>requirements.txt</code>
Le téléchargement de l'étape 5 n'avance plus	Toute la salle télécharge en même temps	Plan B ci-dessous pour la séance, et refaites l'étape 5 chez vous

Plan B — Google Colab, sans rien installer

Si l'installation bloque, **ne restez pas sans rien faire**. Google Colab est un Jupyter qui tourne sur les serveurs de Google : pandas, scikit-learn et matplotlib y sont déjà.

1. Allez sur colab.research.google.com et connectez-vous avec votre compte Google personnel.
2. **Fichier** → **Importer le notebook** : choisissez 01-premier-notebook.ipynb.
3. Icône **dossier** dans la barre de gauche : créez un dossier `donnees`, puis importez-y `ventes-3-agences.xlsx`.

Sur Colab, la cellule 0 affiche ATTENTION : c'est normal, vous n'êtes pas dans l'environnement du cours. **Colab efface vos fichiers à la fin de la session** : avant de partir, **Fichier** → **Télécharger** → `.ipynb`. Colab dépanne une séance ; terminez l'installation chez vous pour les suivantes.

Récapitulatif

Une seule fois : installer uv (étape 2), rouvrir le terminal, placer `ia-elc2` dans votre dossier personnel (étape 3), puis :

```
cd ~/ia-elc2
uv venv --python 3.13
uv pip install -r requirements.txt
uv run jupyter notebook
```

À chaque séance :

```
cd ~/ia-elc2
uv run jupyter notebook
```

Pour arrêter : `Ctrl + C` deux fois dans le terminal.

Sources

- [uv](#) — Installation (documentation officielle)
- [uv](#) — Environnements virtuels
- [Python](#) — Le module `venv`, création d'environnements virtuels
- [PEP 668](#) — Pourquoi les systèmes refusent `pip install`
- [Jupyter Notebook](#) — documentation

Commandes et captures relevées le 30 septembre 2026 sur macOS avec uv 0.11.19, Python 3.13.13, Jupyter Notebook 7.6.3, pandas 3.0.6 et scikit-learn 1.9.1. À cette date, l'installateur officiel fournit uv 0.12.21.